AI, High Performance Computing, and Ethical Considerations

This class gave me more tools as I continue to transition my career from mechanical engineering to data science. Getting experience with common deep learning frameworks such as PyTorch and common tools used in the field such as Python, Pandas, NumPy, Seaborn, Matplotlib, and Scikit-learn. Additionally having in depth discussions about how ethics plays a role in AI, and how as a future data scientist I can combat common problems such as data bias.

Develop and Analyze a GAN and Classifier

Overview

The primary objective of this project was to develop a generator and classifier using the MNIST dataset (70,000 handwritten digit images) in PyTorch. The goal was to build a CNN-based classifier with over 70% accuracy for digit classification and create a GAN-based generator capable of producing lifelike handwritten digits, implement parallel processing techniques to optimize training across multiple CPU and GPU cores. Performance was analyzed using key metrics, and a GIF was generated to visualize the generator's progress throughout training.

Methodologies Used

Model Development

Neural Networks

Classifier: A CNN was designed to classify the MNIST dataset. The model consisted of convolutional layers, max-pooling layers, and fully connected layers, with ReLU activation functions to introduce non-linearity.

Generator: The generator used transposed convolutional layers to upsample random noise into images.

Discriminator: The discriminator was trained to differentiate between real images from the MNIST dataset and fake images generated by the generator.

Data Parallelization

The project utilized PyTorch's DataParallel module to distribute the training process across multiple GPUs, improving training efficiency.

Automatic Mixed Precision (AMP) was employed to optimize memory usage and speed up training by using lower precision (float16) where possible.

Model Evaluation

Performance Metrics

Classifier: Accuracy, precision, and recall were used to evaluate the classifier's performance.

Generator and Discriminator: Binary cross-entropy loss was used to measure the performance of both the generator and discriminator.

Visualization

A GIF was created to visualize the generator's progress over time, showing how the generated images improved throughout the training process.

Matplotlib was used to plot the loss curves for both the generator and discriminator, as well as the precision and recall metrics for the classifier.

Tools and Technologies

Tool	Use
PyTorch	The primary deep learning framework used for building and training the neural networks.
Torchvision	Used for loading and transforming the MNIST dataset.
Matplotlib	For visualizing the generated images, loss curves, and precision/recall metrics.
IPython	For displaying the training progress GIF in a Jupyter Notebook.
NumPy	For array manipulation and data processing.
Scikit-learn	For calculating precision and recall scores.
Os and Psutil	For system resource monitoring and managing parallel processing.

Outputs and Visuals

Discriminator

```
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.model = nn.Sequential(
            nn.Conv2d(1, 64, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(64, 128, kernel_size=4, stride=2,
padding=1),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Flatten(),
            nn.Linear(128 * 7 * 7, 1)
        )

    def forward(self, x):
        return self.model(x)
```

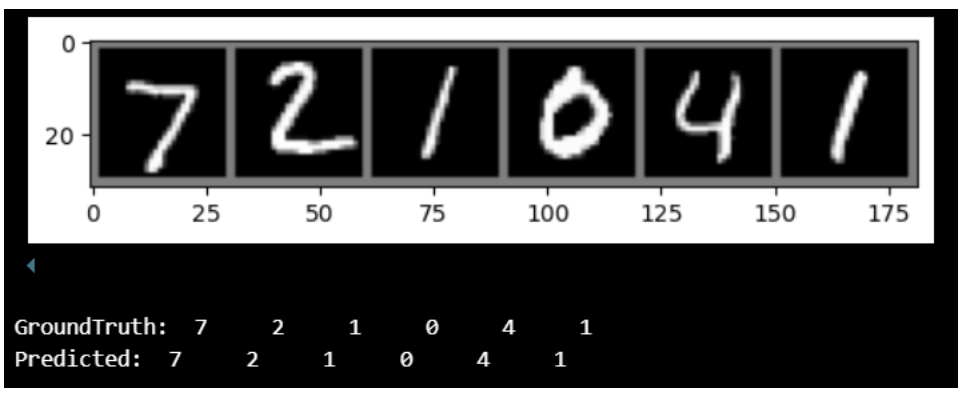
Generator

```
class Generator(nn.Module):
    def __init__(self, noise_dim, img_dim):
        super(Generator, self).__init__()
        self.img_dim = img_dim
        self.fc = nn.Linear(noise_dim, 128 * 7 * 7)
        self.deconv = nn.Sequential(
            nn.ConvTranspose2d(128, 64, kernel_size=4,
stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64, 1, kernel_size=4, stride=2,
padding=1),
            nn.Tanh()
        )

    def forward(self, x):
        x = self.fc(x)
        x = x.view(x.size(0), 128, 7, 7)
        x = self.deconv(x)
        return x
```

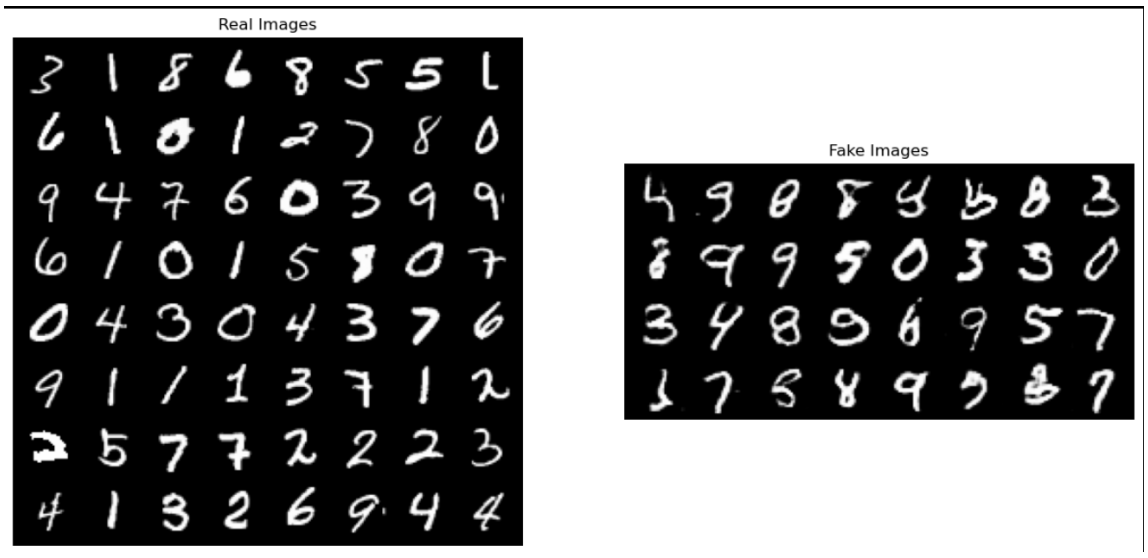
Classifier

```
class Classifier(nn.Module):
    def __init__(self):
        super(Classifier, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3,
stride=1, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.fc1 = nn.Linear(32 * 14 * 14, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = x.view(-1, 32 * 14 * 14)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

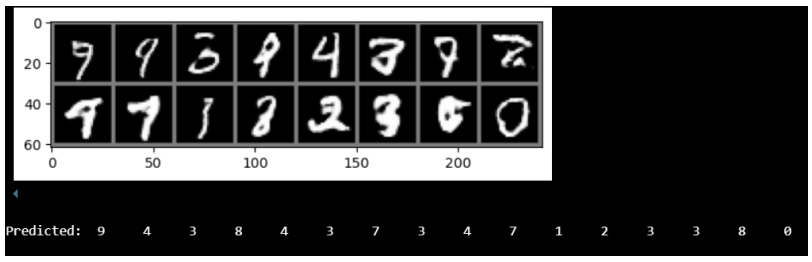
The 3 neural networks used in this project.



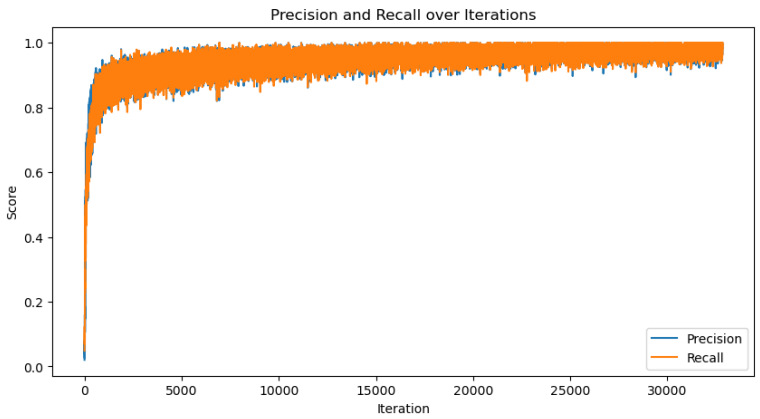
Testing classifier against ground truth labels in MNIST testing dataset.



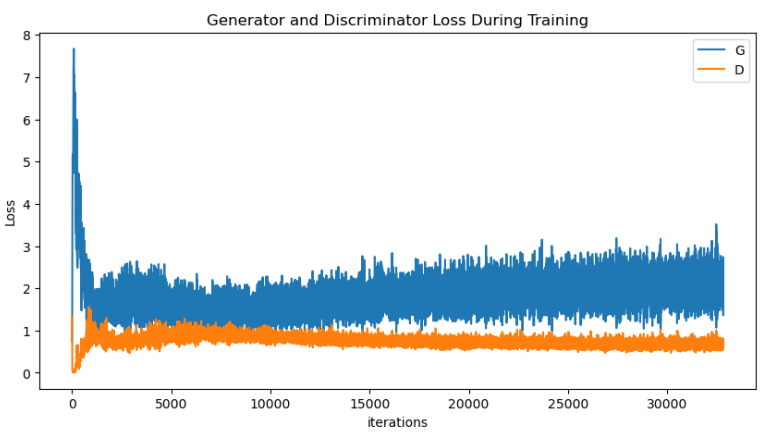
Real images from the MNIST dataset vs synthetic images created by the generator after training.

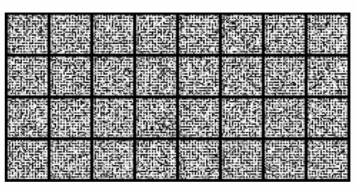


Classifier predicting the digit of synthetic images created by the generator.



Precision and recall of the classifier throughout the training loop.





Generator and discriminator loss throughout the training loop. Gif of the generators progress throughout training by using a fixed noise vector.

Challenges and Solutions

Challenge	Solution
Training GANs and CNNs on large datasets like MNIST can be computationally expensive and time-consuming.	I implemented parallel processing using PyTorch's DataParallel module and Automatic Mixed Precision (AMP) to reduce training time and optimize resource usage.
The generator sometimes produced similar-looking images (mode collapse), failing to generate diverse outputs.	I adjusted the learning rates of the generator and discriminator and experimented with different architectures to stabilize training.

Outcomes and Learnings

The classifier achieved an overall accuracy of 97%, with individual class accuracies ranging from 95.8% to 99.1%. Precision and recall scores were consistently high, indicating that the classifier was effective in identifying handwritten digits.

The generator successfully produced lifelike images of handwritten digits, as evidenced by the decreasing generator loss over time. The discriminator's loss also decreased, indicating that the GAN was converging effectively.

The use of parallel processing and AMP significantly reduced training time, allowing the model to train faster on a multi-core CPU and GPU system.

This project provided valuable insights into the complexities of training deep learning models, particularly GANs and CNNs. The challenges faced during the project, such as mode collapse and overfitting, taught me the importance of hyperparameter tuning and regularization techniques. Additionally, the successful implementation of parallel processing and mixed precision demonstrated the potential for optimizing deep learning workflows.

Develop and Analyze a Linear Regression Model

Overview

The primary objective of this project was to develop and analyze a linear regression model to predict food delivery times based on factors like distance, preparation time, and weather conditions. To evaluate the accuracy, metrics such as R-squared, Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE) were used. The most relevant features were identified through correlation analysis and feature scoring, and visualizations were created like residual plots and histograms to analyze model performance and validate assumptions.

Methodologies Used

Data Preparation

Data Cleaning

Removed irrelevant columns (e.g., Order_ID), handled missing values, and encoded categorical variables (e.g., weather, traffic level) into numerical data using LabelEncoder.

Feature Selection

Used correlation heatmaps and SelectKBest to identify the most significant features influencing delivery times.

Standardization

Normalized the independent variables using StandardScaler to ensure consistent scaling.

Model Development

Linear Regression

Developed a linear regression model using PyTorch, with a neural network architecture consisting of multiple fully connected layers and ReLU activation functions.

Model Training

Trained the model using Adam optimizer and Mean Squared Error (MSE) loss function. Implemented a training loop with hyperparameter tuning to find the best model configuration (e.g., layer sizes, learning rate, batch size, and epochs).

Parallel Computing

Although not explicitly parallelized, the model leveraged PyTorch's efficient tensor operations and GPU acceleration (via CUDA) to speed up training.

Model Evaluation

Performance Metrics

Calculated R-squared, MAE, and RMSE to evaluate the model's predictive accuracy.

Residual Analysis

Created residual plots and histograms to check for linearity, equal variance, and normality assumptions in the model.

Visualization

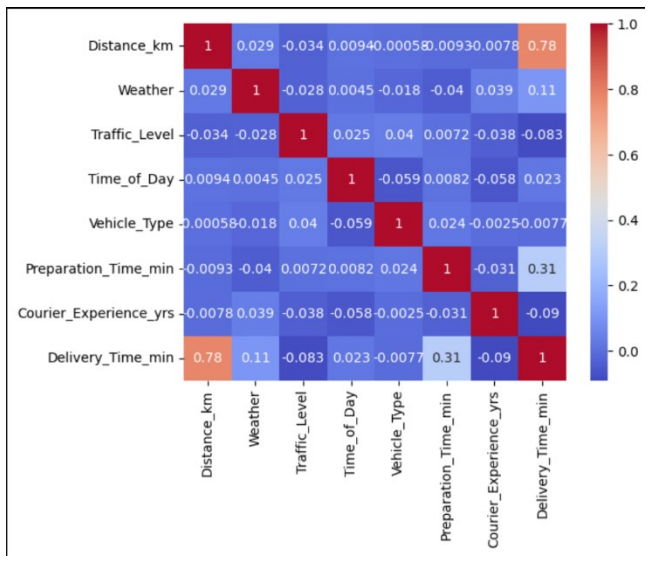
Used Matplotlib and Seaborn to visualize the model's performance, including residual plots and histograms.

Tools and Technologies

Tool	Use
PyTorch	The primary deep learning framework used for building and training the neural network.
Pandas	For data manipulation, cleaning, and data frame creation.
Matplotlib	For visualizing the data, specifically creating histograms and residual plots.
Seaborn	For visualizing the data, specifically creating feature correlation heatmaps.
NumPy	For array manipulation and data processing.
Scikit-learn	Used for data preprocessing (e.g., StandardScaler, LabelEncoder), feature selection (SelectKBest), and model evaluation metrics (e.g., r2_score, mean_absolute_error).
Cuda	Utilized GPU acceleration for faster model training.

Outputs and Visuals

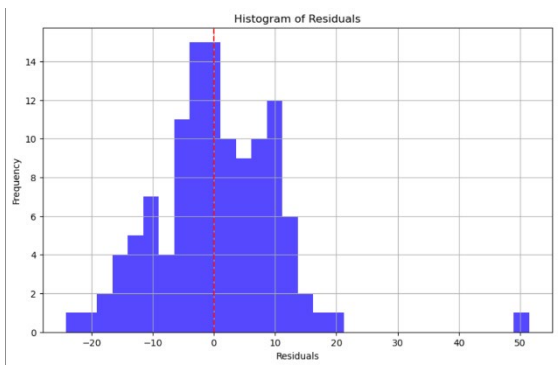
	Feature	Score
0	Distance_km	1482.657919
5	Preparation_Time_min	101.762442
1	Weather	11.350814
6	Courier_Experience_yrs	7.981689
2	Traffic_Level	6.721439
3	Time_of_Day	0.533439
4	Vehicle_Type	0.056719

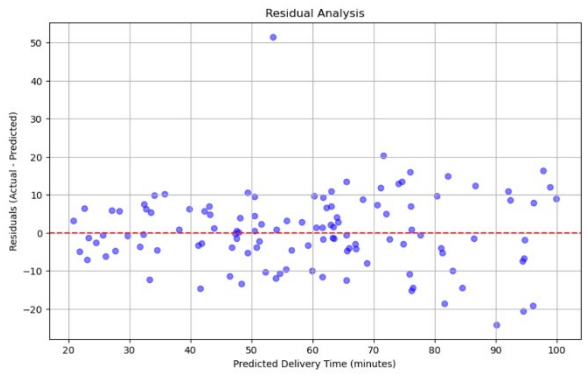


Feature Selection using SelectKBest and seaborn heatmap.

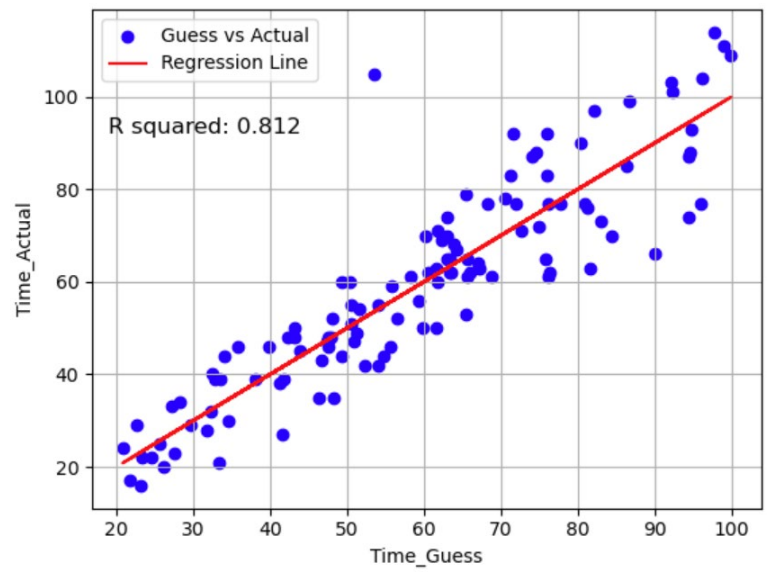
```python
class Time_Predict_Reg(nn.Module):
    def __init__(self, input_size, layer1, layer2, layer3):
        super(Time_Predict_Reg, self).__init__()
        self.linear = nn.Linear(input_size, layer1)
        self.linear1 = nn.Linear(layer1,layer2)
        self.linear2 = nn.Linear(layer2,layer3)
        self.linear3 = nn.Linear(layer3,1)

    def forward(self, x):
        x = torch.relu(self.linear(x))
        x = torch.relu(self.linear1(x))
        x = torch.relu(self.linear2(x))
        x = torch.relu(self.linear3(x))
        return x
```

Custom PyTorch neural network.





Residual Analysis using matplotlib.pyplot to test for linearity, variance, and normality.



Output graph of predictions vs the testing data, including the R squared value and regression line.

Challenges and Solutions

Challenge	Solution
Identifying the most relevant features from the dataset was challenging due to the presence of multiple categorical and numerical variables.	Used correlation heatmaps and SelectKBest to rank features based on their importance, ultimately selecting Distance_km, Preparation_Time_min, and Weather as the most significant predictors.

The model's RMSE (9.85 minutes) and MAE (7.30 minutes) were relatively high, indicating room for improvement.	Analyzed residuals and identified that the model struggled with predicting delivery times over 65 minutes. Recommended removing outliers and experimenting with different model architectures.

Outcomes and Learnings

The linear regression model successfully predicted food delivery times with reasonable accuracy, achieving an R-squared value of 0.812, indicating that it explained 81.2% of the variance in delivery times. The RMSE was 9.85 minutes, and the MAE was 7.30 minutes, suggesting that the model's predictions were, on average, within 7-10 minutes of the actual delivery times. The model performed well for shorter delivery times, but it struggled with longer delivery times.

This project highlights the process of developing and analyzing a linear regression model. It taught me the importance of thorough data preparation and model validation. The project also reinforced my understanding of key regression metrics (R-squared, RMSE, MAE) and the significance of residual analysis in evaluating model performance.

Implementation of a Discriminative Model

Overview

The primary objective of this project was to develop and analyze a discriminative model using a Random Forest Classifier to classify NFL players by their positions (e.g., QB, RB, WR) based on historical season data. The model was developed by using a random forest and then inputting that as an additional feature to a classifier neural network. Model performance was evaluated using a confusion matrix and classification accuracy. Hyperparameter tuning was implemented for the random forest and neural network during training.

Methodologies Used

Data Preparation

Data Cleaning

Removed irrelevant columns (e.g., Player Id, Name), handled missing values, and converted categorical data (e.g., position, receptions) into numerical data using LabelEncoder (Skikit-learn).

Feature Selection

Used correlation heatmaps and SelectKBest to identify the most significant features influencing player positions.

Standardization

Normalized the independent variables using StandardScaler to ensure consistent scaling.

Model Development

Random Forest Classifier

Developed a Random Forest model using scikit-learn, with hyperparameter tuning performed via GridSearchCV to find the best parameters (e.g., n_estimators, max_depth, min_samples_split).

Neural Network

Built a neural network in PyTorch that incorporated the Random Forest's predictions as an additional feature. The network consisted of fully connected layers with ReLU activation functions.

Model Training

Trained the neural network using Adam optimizer and CrossEntropyLoss for multi-class classification. Implemented a training loop with hyperparameter tuning to find the best model configuration (e.g., hidden layer sizes, learning rate, batch size, and epochs).

Model Evaluation

Performance Metrics

Calculated accuracy and generated a confusion matrix to evaluate the model's predictive performance.

Visualization

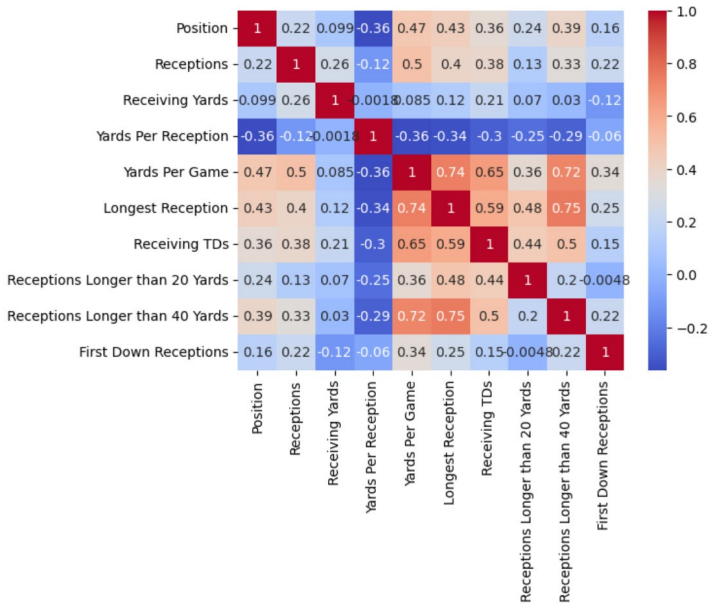
Used Matplotlib and Seaborn to visualize the confusion matrix and analyze the model's classification accuracy.

Tools and Technologies

Tool	Use
PyTorch	The primary deep learning framework used for building and training the neural network.
Pandas	For data manipulation, cleaning, and data frame creation.
Seaborn	For visualizing the data, specifically creating feature correlation heatmaps.
NumPy	For array manipulation and data processing.
Scikit-learn	Used for data preprocessing (e.g., StandardScaler, LabelEncoder), feature selection (SelectKBest), and implementing the Random Forest Classifier.
Cuda	Utilized GPU acceleration for faster model training.

GridSearchCV	For hyperparameter tuning of the Random Forest model.

Outputs and Visuals



Feature	Score
Yards Per Game	540.364324
Longest Reception	436.984787
Receptions Longer than 40 Yards	340.936443
Yards Per Reception	288.006441
Receiving TDs	282.927383
Receptions Longer than 20 Yards	121.516711
Receptions	97.915963
First Down Receptions	52.348952
Receiving Yards	19.012075

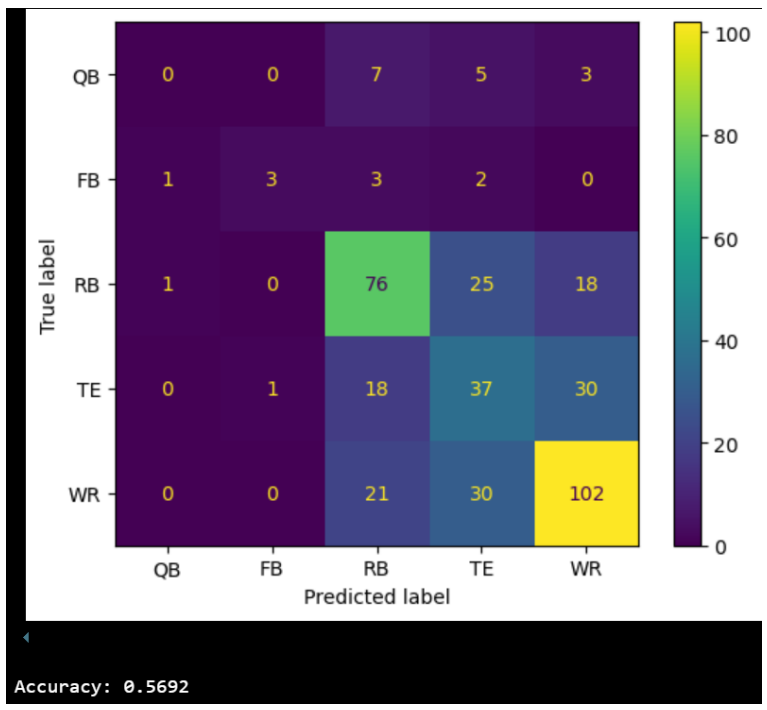
Feature map and feature table to visualize important features for the model.

```
forest_param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [10, 20, 30],
    'min_samples_split': [2, 5, 10]
}
forest = RandomForestClassifier()
grid_search = GridSearchCV(forest, forest_param_grid, cv=5,
scoring='accuracy')
grid_search.fit(x_train, y_train.ravel())
forest = grid_search.best_estimator_
print("Best Random Forest Parameters: ", grid_search.best_params_)
print("Best Random Forest Accuracy: ", grid_search.best_score_)
Max depth: 10, min_samples_split: 5, n_estimators: 100
Accuracy: .557
```

Implementation of the random forest and hyperparameter tuning using a grid search.

```
class NFLModelWithForest(nn.Module):
    def __init__(self, input_size, forest_output_size, hidden_size1,
hidden_size2):
        super(NFLModelWithForest, self).__init__()
        self.fc1 = nn.Linear(input_size + forest_output_size,
hidden_size1)
        self.fc2 = nn.Linear(hidden_size1, hidden_size2)
        self.fc3 = nn.Linear(hidden_size2, 5)
        self.relu = nn.ReLU()

    def forward(self, x, forest_pred):
        x = torch.cat((x, forest_pred.unsqueeze(1)), dim=1)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

Create a PyTorch neural network that uses the forest as a feature.



Confusion matrix that shows the number of correct guesses for each label.

Challenges and Solutions

Challenge	Solution
The dataset had an imbalance in player positions, with some positions (e.g., WR) being overrepresented compared to others (e.g., FB).	Addressed class imbalance by removing outliers and ensuring a more balanced distribution of positions in the dataset.
Tuning hyperparameters for both the Random Forest and the neural network was time-consuming and computationally expensive.	Used GridSearchCV for the Random Forest and implemented a hyperparameter grid search for the neural network to find the optimal configuration.
Identifying the most relevant features from the dataset was challenging due to the presence of multiple categorical and numerical variables.	Used correlation heatmaps and SelectKBest to rank features based on their importance, ultimately selecting key features like Receiving Yards, Receptions, and Touchdowns.

Outcomes and Learnings

The discriminative model successfully classified NFL player positions with reasonable accuracy, achieving an accuracy of 62.3% on the test set. The confusion matrix provided insights into the model's performance, showing that it performed well for some positions (WR) but struggled with others (FB). Next steps would be to continue to address class imbalances in the dataset, along with experimenting with more robust models.

Throughout this project, I was able to get experience in the process of developing and analyzing a discriminative model using both traditional machine learning (Random Forest) and deep learning (neural network) techniques. It showed me the value of proper data preprocessing, and how important the role of feature selection can be. Getting real world experience with unclean data and training neural networks has helped me reinforce what I have learned so far.

Cloud Architecture and Infrastructure

This class was my second in my Master of Data Science program and gave me tools and experience with cloud environments and industry standards and protocols. It also gave me practical experience in designing and deploying infrastructure using Amazon Web Services (AWS). It also provided insight into security protocols and how to apply layers of security in a cloud environment, and how to implement infrastructure as code (IaC) to align with current industry trends and DevOps principles.

Deploy a stack in AWS using IaC and use Serverless Computing to Trigger an Automated Alert

Overview

The primary objective of this project was to automate AWS infrastructure deployment and monitoring using AWS services such as CloudFormation and CloudWatch. Using principles such as infrastructure as code (IaC) and serverless computing to create a scalable, automated system that logs EC2 instance terminations and triggers an email notification via an alarm.

Methodologies Used

<u>Infrastructure as Code (IaC)</u>

AWS CloudFormation templates were created and used to define and deploy infrastructure components. The templates were written in YAML, with specific parameters defined to add additional flexibility.

<u>Event-Driven Automation</u>

Using serverless computing, a function was created to log EC2 termination and to trigger that function to run when the state of an EC2 was changed.

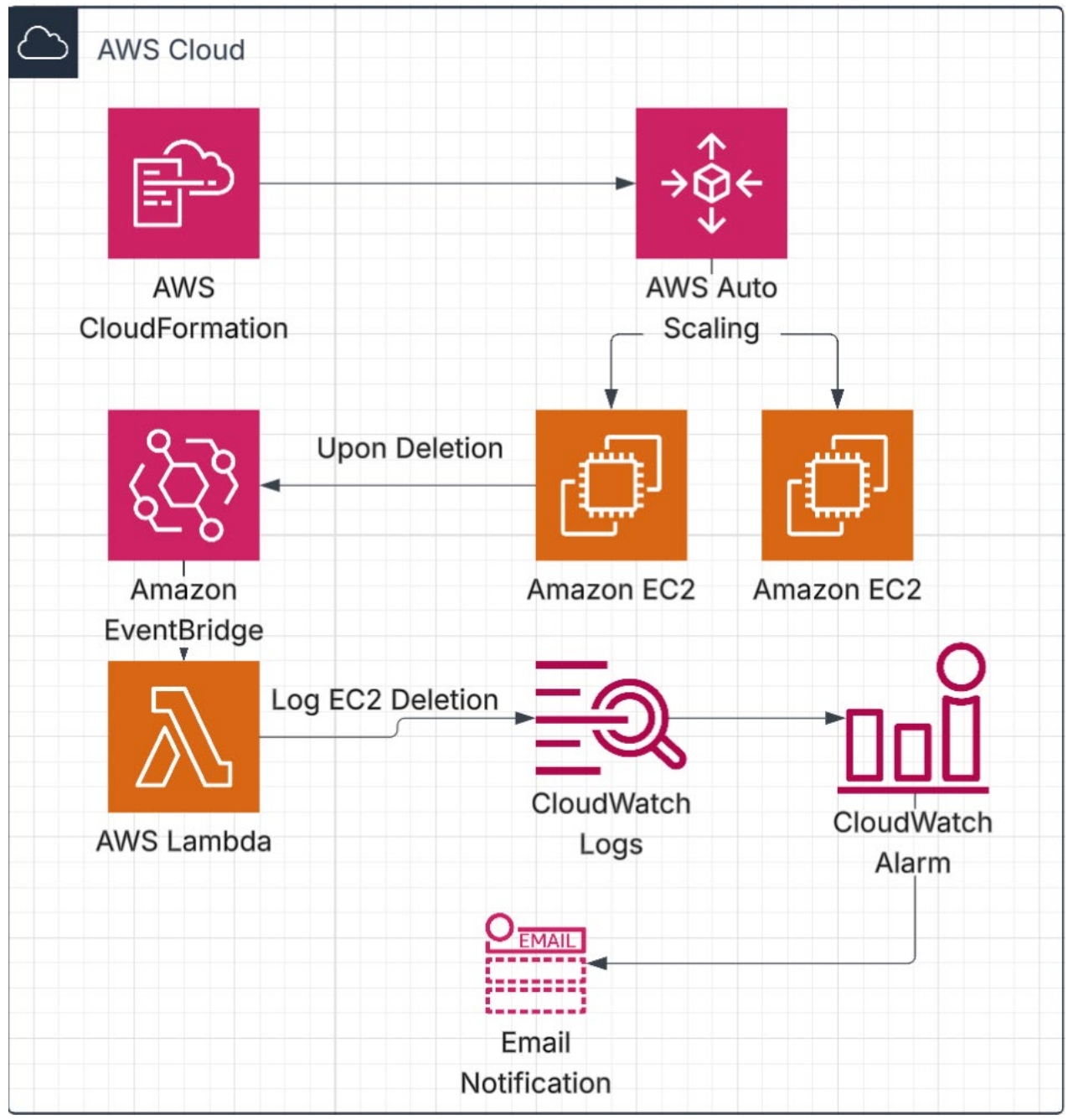
<u>Continuous Monitoring and Alerts</u>

Used built in AWS tools to monitor and create alerts based on log changes and updates. These alerts were sent via email.

Tools and Technologies

Tool	Use
AWS CloudFormation	IaC template for infrastructure deployment.
AWS Lambda	Serverless function to process EC2 termination.
AWS EventBridge	Automated trigger to run the Lambda function upon an EC2 state change.
AWS CloudWatch	Logs that were updated upon the function being triggered and alarms that sent email notifications when a metric crossed a set threshold.
AWS Auto Scaling	Managed EC2 instance scaling that was deployed via the CloudFormation template.

Outputs and Visuals



AWS Architecture Diagram of the project.

Challenges and Solutions

Challenge	Solution
CloudWatch alarms can be over tuned and overly sensitive.	I designed the metric used for the alarm with this in mind, to ensure that the alarm only triggered when I needed to be aware of it. In a business setting there would be KPIs that are defined by the organization.
Manual retrieval of subnet ids for IaC can lead to copy paste and other human errors.	By defining parameters in the template, I was able to select the proper subnet upon deployment of the stack, avoiding needing to copy and paste specific values. This also leads to more secure code as there are not sensitive values in the code itself.

Outcomes and Learnings

I was successful in deploying an automated system to log and alert EC2 terminations and showed the ability to implement end-to-end integration of AWS services for IaC and continuous monitoring.

This project allowed me to get hands on experience with infrastructure as code, as well as serverless computing and continuous monitoring. All of these methodologies are widely used in industry and this project showed me the value they can bring to an organization. Serverless computing can automate more of the basic tasks, freeing up resources to tackle more complex deployments and architectures. Continuous monitoring serves to improve things such as availability and reduce potential downtime by notifying teams when parts of the environment are not working as intended. It is also is a security principle as well as a part of incident response and disaster recovery.

Deploy a Simple Web Application Using AWS

Overview

The objective of this project was to deploy a web application using AWS services. The project goes over setup of a virtual private cloud (VPC), elastic compute cloud (EC2) instance that acts as the host, simple storage service (S3) to act as object storage, relational database service (RDS) for creating a database, and CloudWatch for monitoring.

Methodologies Used

Automation

Using automated backups in RDS along with CloudWatch custom dashboards and alarms to automatically send alerts based on pre-defined KPIs.

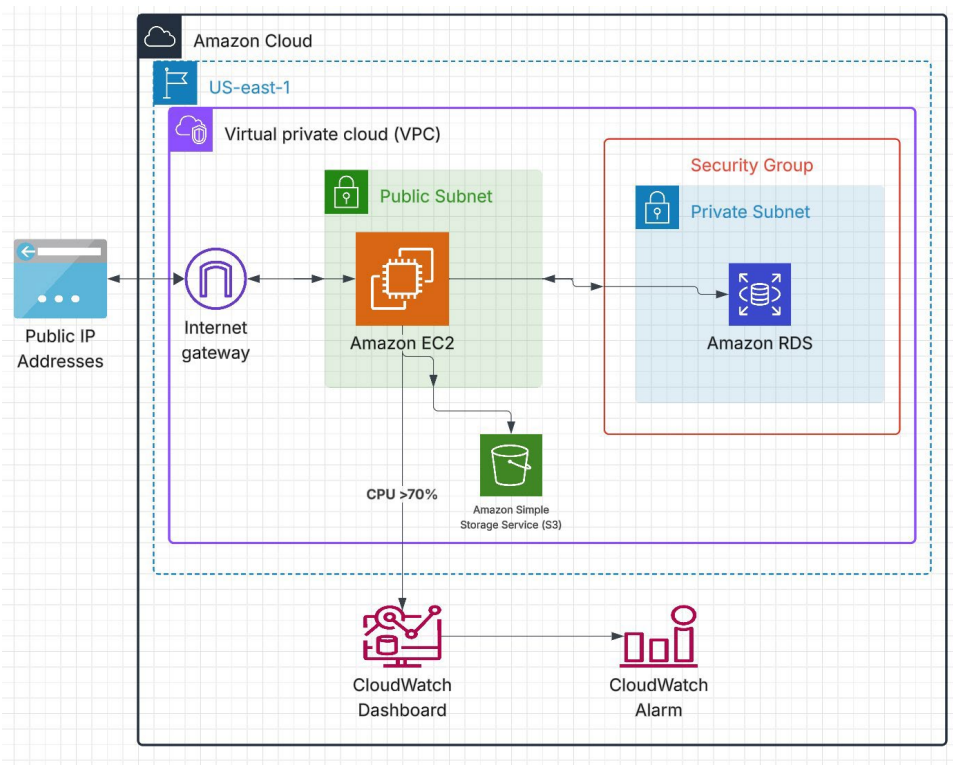
Modular Architecture Design and Implementation

All AWS components (VPC, EC2, S3, RDS, and CloudWatch) were defined, configured, and implemented independently. Security layers were added during configuration of each architecture. These layers included things such as subnets, subnet groups, Identity Access Management (IAM), and security groups.

Tools and Technologies

Tool	Use
AWS VPC	Isolated networks within the AWS cloud where all of our architecture was deployed. Can be duplicated in different regions for redundancy, latency, and compliance.
AWS EC2	Virtual server in AWS. Used to host our Apache Web Server for the web application.
AWS S3	Object storage bucket in AWS. S3 has versioning, encryption, and was set up with read only access from the EC2.
AWS RDS	MySQL database that was hosted in a private subnet in the VPC for security, so other IP addresses cannot access it directly.
AWS CloudWatch	Continuous monitoring tool in AWS that was used to create a custom dashboard for the web app and alarm that both monitored CPU usage of the EC2.
IAM	Used IAM to limit the EC2 to read only access of static storage (S3).
SSH Protocol	Used to connect to the EC2 to deploy the Apache Web Server.

Outputs and Visuals



AWS architecture diagram of the project. It contains all the architecture that was deployed along with additional items such as an internet gateway, subnets, and a security group.

Challenges and Solutions

Challenge	Solution
Ensuring that other IP addresses cannot access the database or data directly.	I created a custom security group to only allow my ip address to access these items. I also implemented an IAM role for the EC2 so it only had read access to the data (S3). In addition, I created a route table to restrict public access to the private subnet where the database was stored.

Outcomes and Learnings

I was able to successfully deploy a web app with secure and monitored AWS infrastructure. This was done by using common methods and tools such as SSH protocol to connect to the virtual instance and deploy an Apache web server.

Some key learning from this project were that you need to have multiple layers of security implemented into your environment. Combining VPCs with security groups, subnets, and IAMs are all best practices and important to keep your data secure. In addition to that, continuous monitoring provides an additional layer of security with automated alerts and alarms when infrastructure falls outside of defined KPIs. Some automation was implemented in this project in the form of the alerts and alarms, but automation overall can reduce operational overhead, especially with cloud service providers where you only pay for code execution times.

Database Systems

This class provided me with knowledge and background of selecting, implementing, and performing CRUD operations on database systems for different business requirements. It gave me hands on experience with several database types including traditional relational databases, NoSQL Key Value and Graph Databases, and Time Series Databases. It gave me insight into different data structures and how they are stored in different types of databases.

Designing an AI Enhanced Database Ecosystem Using AWS for a Specific Business Case

Overview

The objective of this project was to design a database ecosystem with AI capabilities for a given business scenario. The project was to perform a case study on a business and decide what database types to use given the issues and goals of the company. The project goes through the selection, implementation, and basic queries that could be used to meet the company's goals, along with how AI could be integrated into the ecosystem.

Methodologies Used

<u>Schema Design</u>

Using tools like Lucid, Entity relationship and architecture diagrams were created to demonstrate my ability to effectively design a database schema.

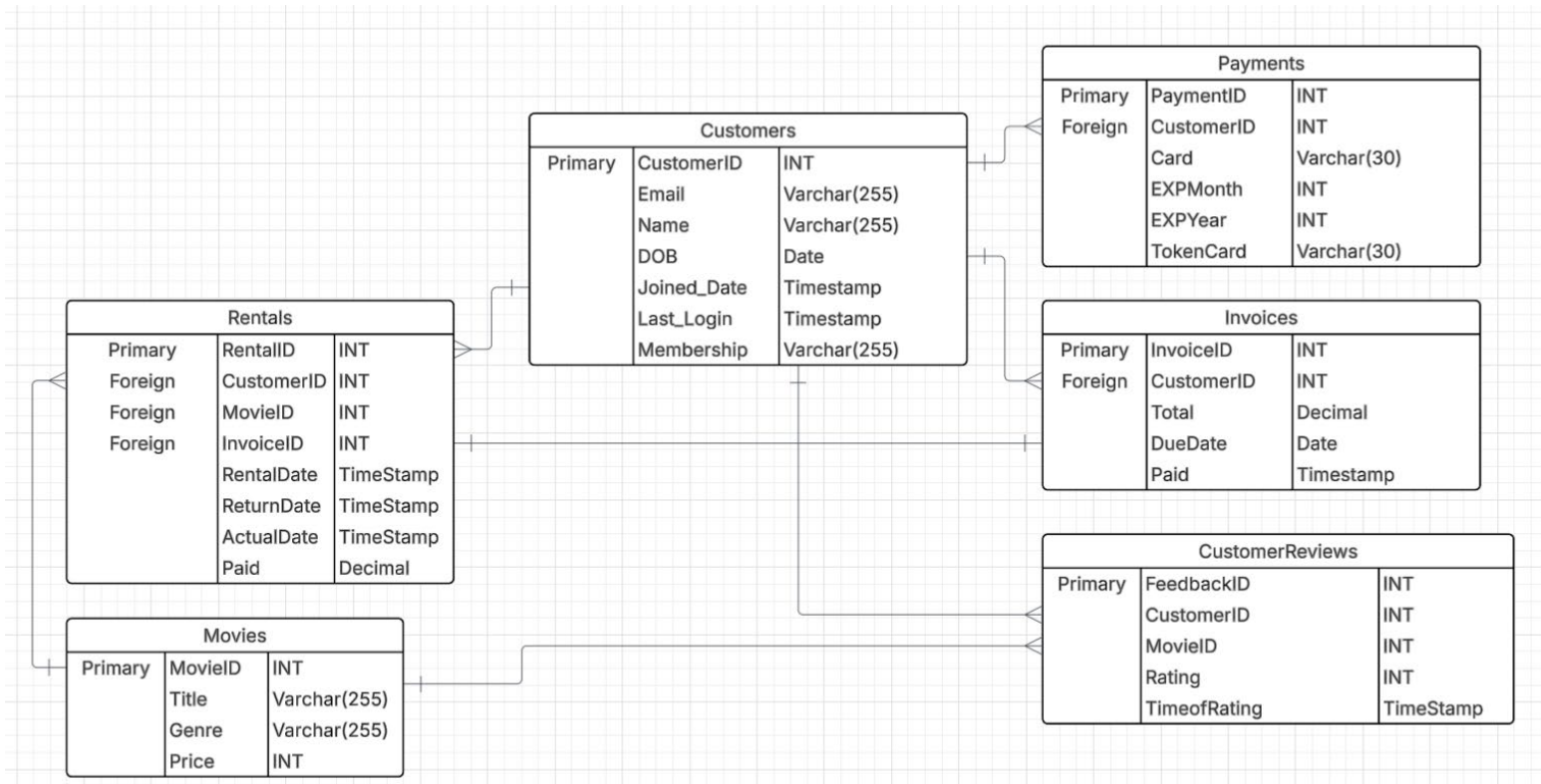
<u>Database Design and Implementation</u>

The databases were designed and developed to meet the company's specific needs and were implemented using Amazon's SDK for Python using the AWS CLI and the Boto3 library.

Tools and Technologies

Tool	Use
Amazon SDK	To deploy AWS infrastructure from my local code in python using the boto3 library.
AWS Timestream	Time series database that was used to help the company perform real time analysis and historical analysis of trends in data.
AWS DynamoDB	Key Value NoSQL database that was used to store the unstructured data that the company needed.
AWS RDS	Postgre database that was hosted in a private subnet in the VPC for security, so other IP addresses cannot access it directly.
AWS Neptune	NoSQL graph database that was used to model and visualize the relationships present in the RDS.
IAM	Used IAM to limit access to the databases hosted in the private subnet.
AWS Elasticsearch	Used to provide a more in-depth search feature for the company's app users.
NoSQL Workbench	Used to create a visual representation of the key value database table design.

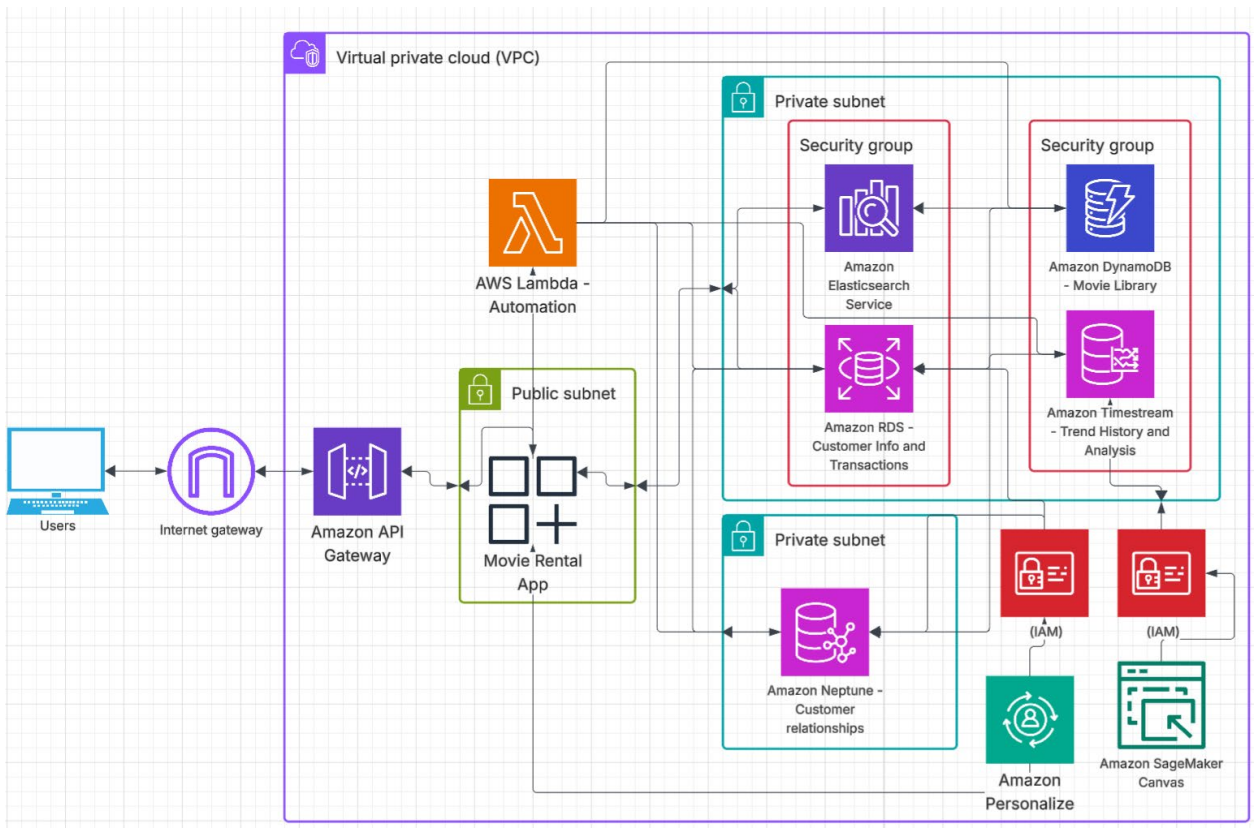
Outputs and Visuals



The entity relationship diagram that was created for the RDS that was implemented into the ecosystem.

Primary key		Attributes			
Partition key: MovieID	Sort key: MovieTitle	Director	Genre	Year	Cast
3cayfhK(c3	m2'QR>.G'h	Zc?zVpc`e3	["Giraffe","Hippo","Zebra"]	2669071199895552	["Giraffe","Hippo","Zebra"]

The NoSQL Key value store Table that was implemented into the ecosystem. It also includes two global secondary indexes for faster queries based on theoretical access patterns.



AWS architecture diagram of the project. It contains the databases used along with the AWS AI tools that would be integrated into the ecosystem and some of the security protocols that were used.

Challenges and Solutions

Challenge	Solution
Determining what data needed to be migrated to new databases.	I chose only to move some of the data into new databases, and kept a chunk of it in a RDS. This not only made the most sense given the problem statement, but it also would keep the project costs down as well.

Outcomes and Learnings

I was able to successfully create a database ecosystem that met the company's needs. The company was a movie rental company and needed to create an ecosystem that allowed for AI integration and eliminated problems with their different data needs and structures. My solution was to move their movie library to a NoSQL database for its benefits with a flexible schema and ability to handle the unstructured nature of the data.

I kept their customer information and billing information in a relational database, not only for ACID compliance but also due to the rigid nature and less variability of customer data. I added a graph database to visualize the relationship between movie's being rented and customers, and a time series database to make analysis of trends over time easier as well. Overall, this project reinforced the concepts of database selection and design for me and gave me a real-world example to work through as if I was a database administrator.

Designing and implementing a Time Series Database for DevOps Monitoring

Overview

The objective of this project was to design and implement a time series database (AWS Timestream) to successfully monitor DevOps architecture for a given business scenario. The project called for an analysis of the company's problems in DevOps architecture and how a time series database can alleviate these issues. The assignment goes through an overview of DevOps monitoring, time series databases, implementation, and sample queries that could be used in a real-life situation.

Methodologies Used

Database Design and Implementation

The database was designed and developed to meet the company's specific needs and were implemented using Amazon's SDK for Python using the AWS CLI and the Boto3 library.

Tools and Technologies

Tool	Use
Amazon SDK	To deploy AWS infrastructure from my local code in python using the boto3 library.
AWS Timestream	Time series database that was used to help the company perform real time analysis and historical analysis of trends in data.
AWS CLI (Command Line Interface)	This was used to enable local access by providing a key generated for a user profile in AWS Identity Access Management.
Python	This was used to deploy the Timestream architecture from my local instance using the AWS SDK.

Outputs and Visuals

Sample Data for Test Queries

- The following code was used in python to upload a test data set I created in a csv file to test writing data into Timestream.
  - Note that if my timestamps were not within the in-memory store retention limits, this data would be rejected if I had not enabled magnetic storage writes in Timestream.

```
with open(csv_file_path, 'r', encoding='utf-8-sig', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    print("CSV Headers:", reader.fieldnames)
    for row in reader:
        dimensions = [
            {'Name': 'Application_ID', 'Value': row['Application_ID']},
            {'Name': 'Application_Name', 'Value': row['Application_Name']}
        ]
        metrics = ['CPU_Usage', 'HTTP_Status', 'Network_Throughput',
'Memory_Usage']
        records = []
        for metric in metrics:
            value_type = 'DOUBLE' if metric != 'HTTP_Status' else 'BIGINT'
            records.append({
                'Dimensions': dimensions,
                'MeasureName': metric,
                'MeasureValue': row[metric],
                'MeasureValueType': value_type,
                'Time': parse_timestamp(row['Time_Stamp']),
                'TimeUnit': 'MILLISECONDS'
            })
```

1

```
try:
            timestream.write_records(
                DatabaseName=database_name,
                TableName=table_name,
                Records=records
            )
            print(f"Uploaded metrics for {row['Application_ID']} at
{row['Time_Stamp']}")
        except Exception as e:
            print(f"Error uploading record: {e}")
            print(e.response)
            time.sleep(1)
```

2

One of the slides that shows the python code used to upload sample DevOps data into Timestream from a local instance.

Timestream vs Traditional DBs

Comparison for Time-Series Data Use Case	Timestream	Traditional Relational DB
Scalability	Hosted on AWS, can scale horizontally as needed as more data gets collected and ingested.	Scales vertically by adding additional hardware, which can be costly. Requires additional software and architecture to scale horizontally using partitions.
Storage	Has two dedicated memory layers, so data can be stored in memory for lower latency and quicker query access for real time analytics.	Stores everything in magnetic disk storage, which has high latency due to the I/O requirements for data retrieval from the disk.
Data Ingestion	Timestream has specific architecture and design to support large scale data ingestions at a time.	Can struggle with applications that require low latency and high write throughput, such as DevOps and IoT.
Retention	Has built in capability to set time to live (TTL) polices for data in memory and in magnetic storage.	Needs additional setup to properly implement retention polices for the data.

(Compare Amazon Timestream for LiveAnalytics vs SQL Server, 2017)

One of the slides in the project that compares Timestream to a traditional relational database.

Challenges and Solutions

Challenge	Solution
Getting a connection between the AWS management console and my local machine for deploying infrastructure and other code remotely.	I used the provided SDK from Amazon and AWS CLI to configure a proper connection between my local computer and the AWS management console. This allowed me to execute python files for deployment and other operations.

Outcomes and Learnings

I was able to successfully analyze the business case and show the benefits of a Time series database for DevOps monitoring. The problems that the company was having was with both real time monitoring for things like anomaly detection and scaling of infrastructure and historical analysis to make better business decisions on their infrastructure needs. By implementing AWS Timestream, we were able to mitigate these problems. Timestream has an in-memory storage layer, which is critical for reducing latency to perform the real time analytics that were needed to monitor the infrastructure. Timestream also has a magnetic storage layer, which is great for performing the historical analysis over a larger period. Timestream also can scale up as needed as the data volume increases over time. This project gave me hands on exposure to the methodology and implementation of time series databases and their use cases.

Enterprise Architecture, Strategic and Technical Research

This class provided me with tools to communicate effectively to leadership teams and other key stakeholders in project development situations such as digital business transformations. I got experience working with several different enterprise architecture frameworks, such as TOGAF (The open group architecture framework), Zachman, Zero Trust, and others. I also created plans and performed risk analysis in real world business scenarios.

Creating a Unified Multi-Domain Enterprise Architecture

Overview

The goal of this project was to create an enterprise architecture plan that formed a unified user experience for the mock company AlphaCorp. AlphaCorp had three main business units of Healthcare, Fintech, and E-commerce, and their main goals were to become an industry leader in cybersecurity, optimize operational costs, and scale operations effectively to support ongoing growth.

Methodologies Used

TOGAF (The Open Group Architecture Framework)

The plan was put together using ADM (architecture development method) and TOGAF principles, especially for creating the business and technology maps, and data flow diagrams.

Zero Trust Architecture

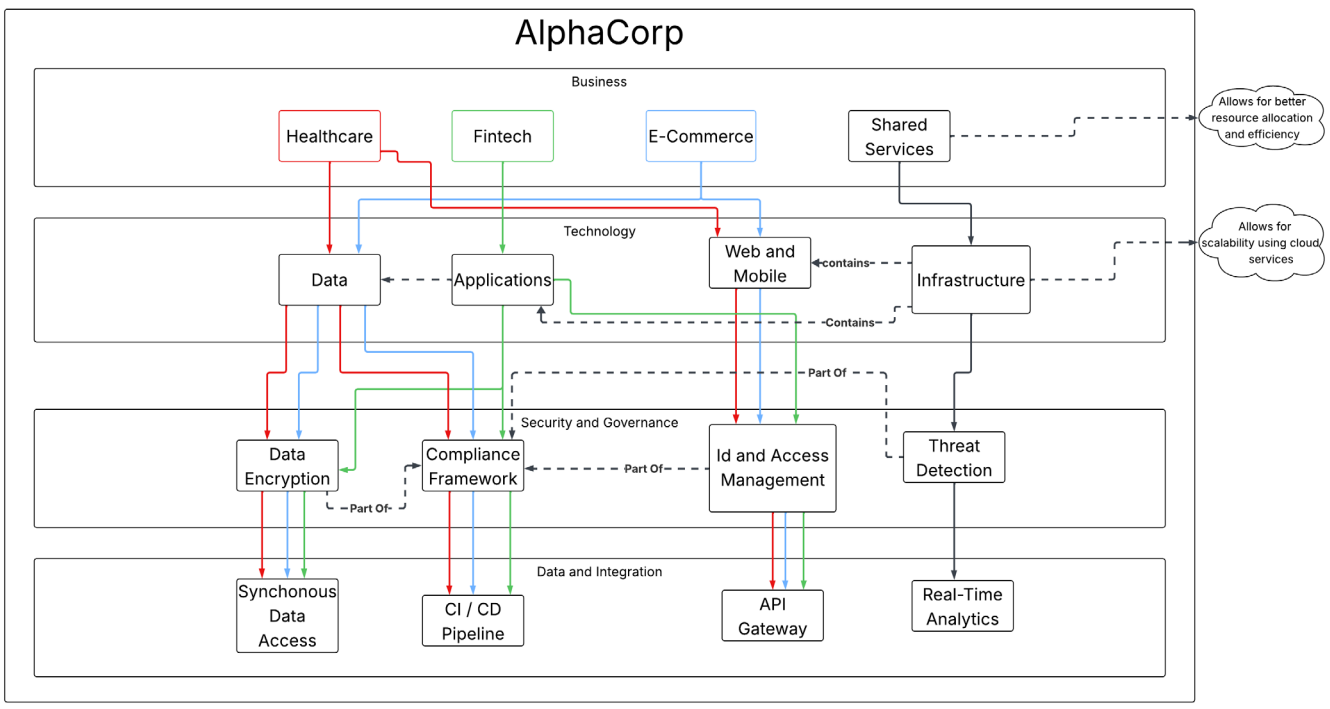
Zero Trust Architecture principles were applied during the cyber security analysis portion of the project. It was used to analysis the risks and prioritize investments based on how critical each risk was and the potential impact of each risk.

Tools and Technologies

Tool	Use

Lucid	To create process maps and data flow diagrams.
Microsoft PowerPoint	To create the presentation and provide additional visuals to the project.

Outputs and Visuals



A diagram created in Lucid to highlight the new unified architecture plan for AlphaCorp.

Inventory – Cyber Asset

Asset #	Name	Location	Ownership	Configuration	Criticality
A1	Firewall	Branch Offices	Cyber Security Team	Outdated technology	High
A2	Servers	On premise	IT Architecture Team	Legacy	High
A3	E-commerce site	Cloud (public)	E-commerce division IT	Newest technology and software on public cloud.	High
A4	Computers and laptops	Employee operated (global deployment)	IT	Imaged by AlphaCorp IT.	Medium
A5	Patient Records	Cloud (private)	Healthcare Division IT	Software on private cloud. HIPAA compliant.	High
A6	Payment Processing	Cloud (public)	Fintech Division IT	Newest technology and software on public cloud.	High

Inventory – Information Asset

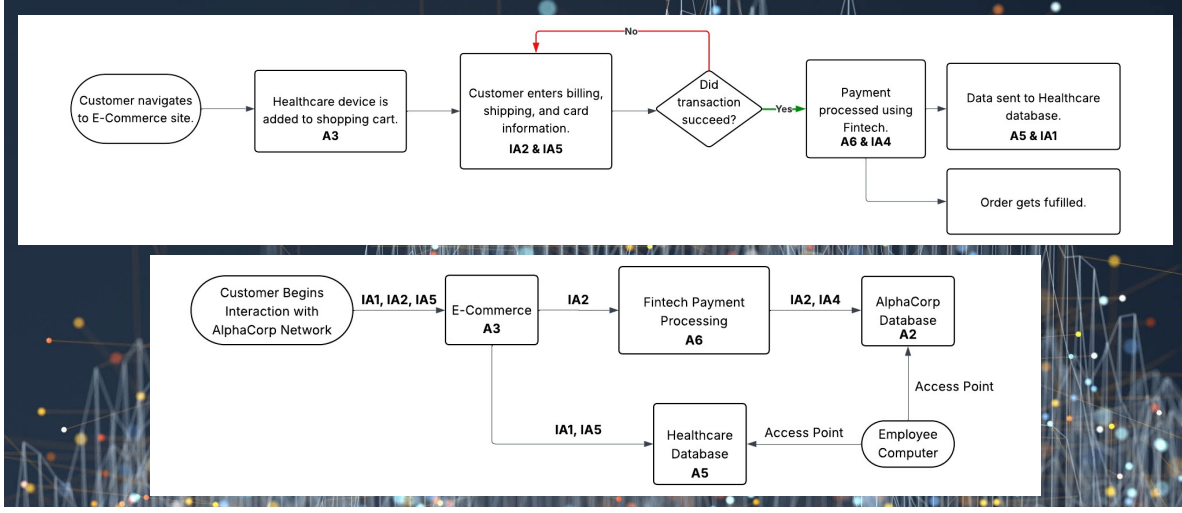
Asset #	Data Type	Owner	Classification	Linked Cyber Asset
IA1	Patient Health Information	Healthcare Division	Restricted and Confidential	A1 A5
IA2	Credit Card Payment Info	Fintech Division	Restricted and Confidential	A6
IA3	AlphaCorp Intellectual Property	AlphaCorp Leadership	Restricted	A1 A4
IA4	AlphaCorp Financial Records	AlphaCorp Leadership	Confidential	A1
IA5	Customer Payment Information	E-Commerce Division	Confidential	A3

Compliance Assessment

Requirement	Asset / Process	Status	Gap
GDPR (E-Commerce)	IA2 - Credit Card Payment Info IA5 - Customer Payment Information A3 – E-commerce site	Partial	Data integrity concerns include vulnerabilities in the system. No documented policies for notifying of a data breach.
HIPPA (Healthcare)	IA1 - Patient Health Information A5 - Patient Records system	Partial	Legacy servers combined with no auditing process lead to partial noncompliance.
PCI-DSS (Fintech)	A6 - Payment Processing IA2 - Credit Card Payment Info IA5 - Customer Payment Information	Compliant	End-to-end encryption ensures compliance with this sensitive financial data.
NIST (Security Framework)	A1-A6 IA1-IA5	Non-compliance	No consistent penetration testing coupled with outdated infrastructure and no continuous monitoring processes in place lead to non-compliance here.

Slides showing the inventory and compliance assessments for the cyber security analysis.

Business Process Map and Data Flow Diagram



The Slide shows the business process map and data flow diagrams for our cyber security analysis.

Risk #	Vulnerability	Asset #	Likelihood	Impact	Level	Current State
R1	Firewall is outdated.	A1 - Firewalls IA1 - Patient Health Information IA5 - Customer Payment Information	High	Theft of sensitive financial and health data. Insertion of ransomware into AlphaCorp's network.	High	Basic filtering of access based on legacy technology.
R2	No continuous training of staff in security protocols.	A4 - Computers and laptops IA1 - Patient Health Information IA2 - Credit Card Payment Info IA3 - AlphaCorp Intellectual Property IA4 - AlphaCorp Financial Records IA5 - Customer Payment Information	Medium	Opens up AlphaCorp to internal threats. Also has the potential for malware and stolen data such as IA3.	Medium	Organization currently deploys two factor authentication policies on all devices as well as providing initial training for employees.
R3	Legacy servers without security patches.	A2 – Private Servers IA1 - Patient Health Information IA3 - AlphaCorp Intellectual Property IA4 - AlphaCorp Financial Records	High	Data breach or architectural failure leading to government fines, financial loss, and loss of customer trust.	Critical	None

R4	No consistent schedule for penetration testing.	A3 - E-commerce site A5 - Patient Records A6 - Payment Processing IA1 - Patient Health Information IA2 - Credit Card Payment Info IA5 - Customer Payment Information	Medium	Potential for different cloud server configurations that have different security umbrellas in different domains, leading to noncompliance and vulnerabilities.	Medium	AlphaCorp has an end-to-end encryption policy to protect data.

Table highlights the risks in the current cyber security umbrella.

Priority Level (1 – high, 5 – Low)	Description	Business Goal	Risks	ROI
1	Update legacy systems and migrate data to cloud infrastructure.	Cost optimization and scalability.	R3	High ROI. Updating these systems and migrating data to the cloud mitigates the most critical risk area that AlphaCorp has. Additionally it provides cost savings through consolidation of data storage as well as the inherent scalability of a cloud service provider.
1	Update firewall hardware and technology.	Cybersecurity enhancement	R1	High ROI. Updating the firewalls lessens the likelihood of ransomware, malware, and other unauthorized access to core data.
2	Continuous monitoring and regular penetration testing.	Cybersecurity enhancement and compliance.	R4	Med ROI. Continuous monitoring allows the org. to notice and patch vulnerabilities before they happen reducing the potential of negative financial impact.
2	Zero Trust Architecture model for cyber security.	Cybersecurity enhancement	R1 R2 R4	Med ROI. Adopting this philosophy will provide long-term benefits of a secure network and infrastructure for AlphaCorp and is industry standard.
3	Ongoing security training for employees.	Cybersecurity enhancement and resource allocation.	R2	High ROI. Training the employees to act as another layer of security defense is an easy way to better protect your network without a significant technological investment.

Table shows what cyber security risks need to be invested in the most based on analysis.

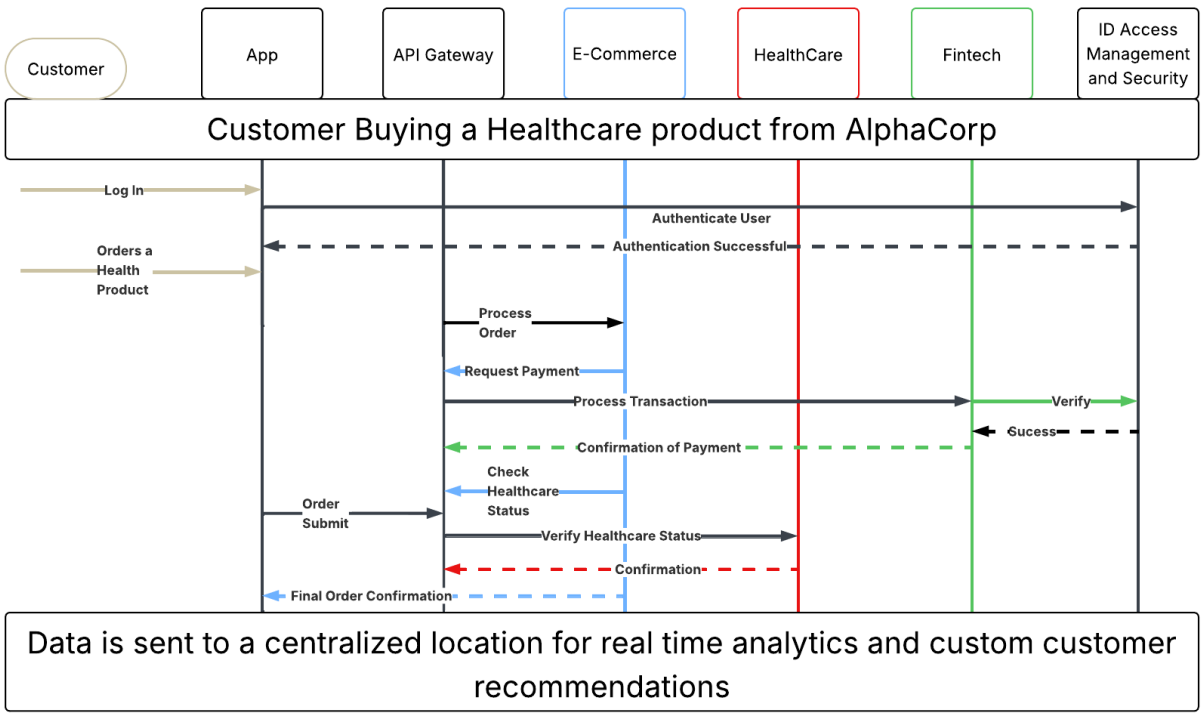


Diagram showing the new unified architecture in action for a mock scenario of a customer buying a healthcare related product within AlphaCorp.

Challenges and Solutions

Challenge	Solution
Transforming theory into practical use and properly applying TOGAF and Zero Trust to the AlphaCorp Scenario.	I used industry best practices to successfully map out business processes and perform an audit of the current state of the organization.

Outcomes and Learnings

I was successfully able to analyze the current state of the organization, as well as provide a thorough analysis of the company's cyber security profile. By implementing a standardized compliance framework, creating infrastructure that is shared among the three main divisions of the company, and creating a migration plan to update legacy infrastructure to cloud service providers, I was able to meet the goals of the company which were to be an industry leader in cybersecurity measures, optimize operational costs, and scale operations effectively to support growth. I learned how to diagram effectively and perform basic risk analysis and audits of technology and resources by utilizing TOGAF and ADM.

Project Management, Systems Development, and Risk

This class provided me with tools to communicate effectively to leadership teams and other key stakeholders in project development situations such as digital business transformations. I got experience with project management methodologies such as the Spotify model and Sprint. The class also gave me real world experience in dealing with risk, from identification all the way to mitigation.

Creating a Risk Plan for the Integration of AI into an E-commerce Platform

Overview

The goal of this project was to create a risk mitigation plan for the top three risks associated with the integration of AI into a legacy, e-commerce platform. Using industry standard risk methodologies, the top three risks were identified, and a mitigation plan was developed to reduce/eliminate the risks. A risk register was also created to continuously monitor these risks with the others in the project.

Methodologies Used

Risk Matrix

A risk matrix was put together to quantify the potential impact to the company if the risk was realized. This allowed us to numerically prove what the top three risks were in an effort to prioritize them.

NIST AI Risk Management Framework

The NIST AI Risk Management Framework was utilized to develop the identification, and mitigation plans for the top three risks identified. These would be things like privacy by design and implementation of a change control process.

Tools and Technologies

Tool	Use
Lucid	To create process maps and data flow diagrams.
Microsoft PowerPoint	To create the presentation and provide additional visuals to the project.

Outputs and Visuals

Risk	Probability	Impact	Score
R1 - Data Privacy	5	5	25
R2 - Algorithm Behavior	4	5	20
R3 - Integration Complexity	4	4	16
R4 - Scope Creep	4	3	12
R5 - Data Quality	3	4	12
R6 - Security Vulnerabilities	2	5	10
R7 - Model Drift	3	3	9
R8 - Stakeholder Confusion	2	3	6

- Probability: 1 = very low, 5 = Very High
- Impact: 1 = None, 5 = Critical
- Risk Score = Probability x Impact

Probability-Impact Matrix

Impact / Probability	1 (Very Low)	2 (Low)	3 (Medium)	4 (High)	5 (Very High)
5 (Critical)	5	10 (R6)	15	20 (R2)	25 (R1)
4 (Major)	4	8	12 (R5)	16 (R3)	20
3 (Medium)	3	6 (R8)	9 (R7)	12 (R4)	15
2 (Low)	2	4	6	8	10
1 (None)	1	2	3	4	5

- R1 - Data Privacy has the highest risk.
- R2 - Algorithm Behavior, R3 - Integration Complexity ,R4 - Scope Creep, R5 - Data Quality are also high.

Probability-Impact Matrix

Slides showing the risk matrix to numerically prioritize the risks associated with AI integration.

R1 – Data Privacy

- Privacy By Design methodology - Organization
- Enforce encryption, access controls, and data minimization – Security Team
- Up the frequency of data audits – Data Officer
- Design AI models with privacy in mind – ML Engineer
- Resources needed: Compliance software, legal team, training
- Steps: Map all data flows that integrate with AI, Classify and label data appropriately, implement at rest and in transit encryption, validate compliance with legal team for requirements.

Sources: [8,10,13,14]

R2 – Algorithm Behavior

- Implement explainability and interpretability models – ML Engineers
- Perform bias assessments and create validation checkpoints – Quality Assurance Engineers
- Ensure human-in-the-loop oversight of models – Project Manager / Engineering teams
- Resources needed: validation Frameworks, bias detection tools, continuous monitoring buy in
- Steps: create layers within algorithm for explainability, run fairness tests, establish metrics and thresholds for drift and accuracy, deploy dashboards for real time monitoring.

Sources: [8,10,13,14]

R3 – Integration Complexity

- Perform integration testing early and often - DevOps
- Ensure data quality and combability before model deployment - DevOps
- Leverage APIs and middleware to assist with communication between legacy and new AI – Engineering teams
- Resources needed: testing environments, API tools, data monitoring systems, more dedicated resources
- Steps: Audit current data to determine structures, build CI/CD pipelines, create prototypes for integration, implement testing checkpoints throughout development

Sources: [8,10,13,14]

Slides showing the risk mitigation strategies for the top three risks identified. The slides also show the owners of these plans.

Risk ID	Title	Description	Category	Probability	Impact	Score	Mitigation	Owner	Status
R1	Data Privacy	Mismanagement leads to loss of customer trust, regulatory fines and violations, and overall harm to the reputation of the company.	Privacy	5	5	25	Privacy By Design, enforce encryption, access controls, and data minimization.	Data Officer	Open
R2	Algorithm Behavior	Sometimes produce strange or inaccurate results.	Technical	4	5	20	Implement explainability and interpretability models, perform bias assessments and create validation checkpoints, human-in-the-loop	ML Engineering Team	Open
R3	Integration Complexity	Can be compatibility issues with legacy systems.	Technical	4	4	16	Perform integration testing early and often, Ensure data quality and combability before model deployment, leverage APIs and middleware	Systems team	Open
R4	Scope Creep		Project	4	3	12		Project Manager	Open
R5	Data Quality		Data	3	4	12		Data Engineers	Open
R6	Security Vulnerabilities		Cybersecurity	2	5	10		Security Team	Open
R7	Model Drift		Operations	3	3	9		ML Engineering Team	Open
R8	Stakeholder Confusion		Organizational	2	3	6		Business Analyst	Open

The risk register for the project.

Challenges and Solutions

Challenge	Solution
Transforming theory into practical use and properly applying NIST as well as other risk frameworks.	I used industry best practices to successfully identify, prioritize, and mitigate risks associated with this integration effort.

Outcomes and Learnings

I was successfully able to analyze the current state of the organization, as well as provide a thorough analysis of the company's current risks. I then used best practices to identify these risks using a probability-impact matrix. This information was then used to prioritize the risks, with the top three risks being identified as requiring immediate mitigation. Mitigation plans and owners were assigned based on industry standards and successes from other companies. All risks were compiled into a risk register for the purpose of continuous monitoring for the duration of the project cycle. This project taught me how to approach risks and how to put a plan together to successfully address and minimize them.

Machine Learning and Artificial Intelligence

This class provided me with tools to build and expand on my knowledge of machine learning and artificial intelligence models and applications. I got experience with industry standard tools for building traditional and deep learning models using Scikit-Learn and TensorFlow. I also got real world experience in creating real-time data applications and creating dashboards that present the information in easy to digest ways.

Building a Deep Learning LTSM Model in TensorFlow

Overview

The goal of this project was to create a deep learning LTSM model using TensorFlow. Using historical data for the Yahoo stock price, a model was created to predict the price for the next day. Initial data was analyzed and feature engineering was performed and variable correlation was identified. A neural network was created with five layers to train on 1400 historical data points.

Methodologies Used

Exploratory Data Analysis (EDA)

The training data was put into a pandas dataframe to explore the dataset. The shape of the dataset, missing values, the datatypes, and common statistical values such as std and mean were compiled and outputted. Feature correlations were also determined at this step.

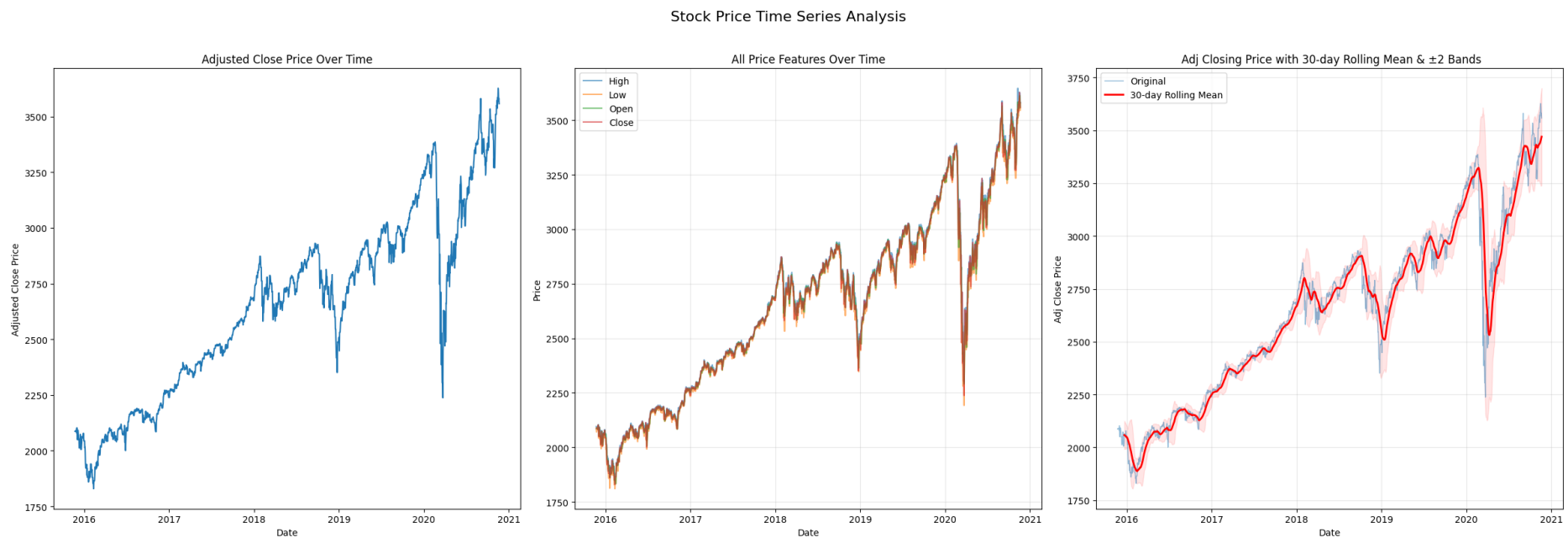
Model Training and Building

The training loop was created by splitting the historical data into an 80% training / 20% testing split to avoid overfitting the model. Standard practices of training in batches and using multiple epochs. Functions were created to ensure that the model is trained in the same way every time new data comes in.

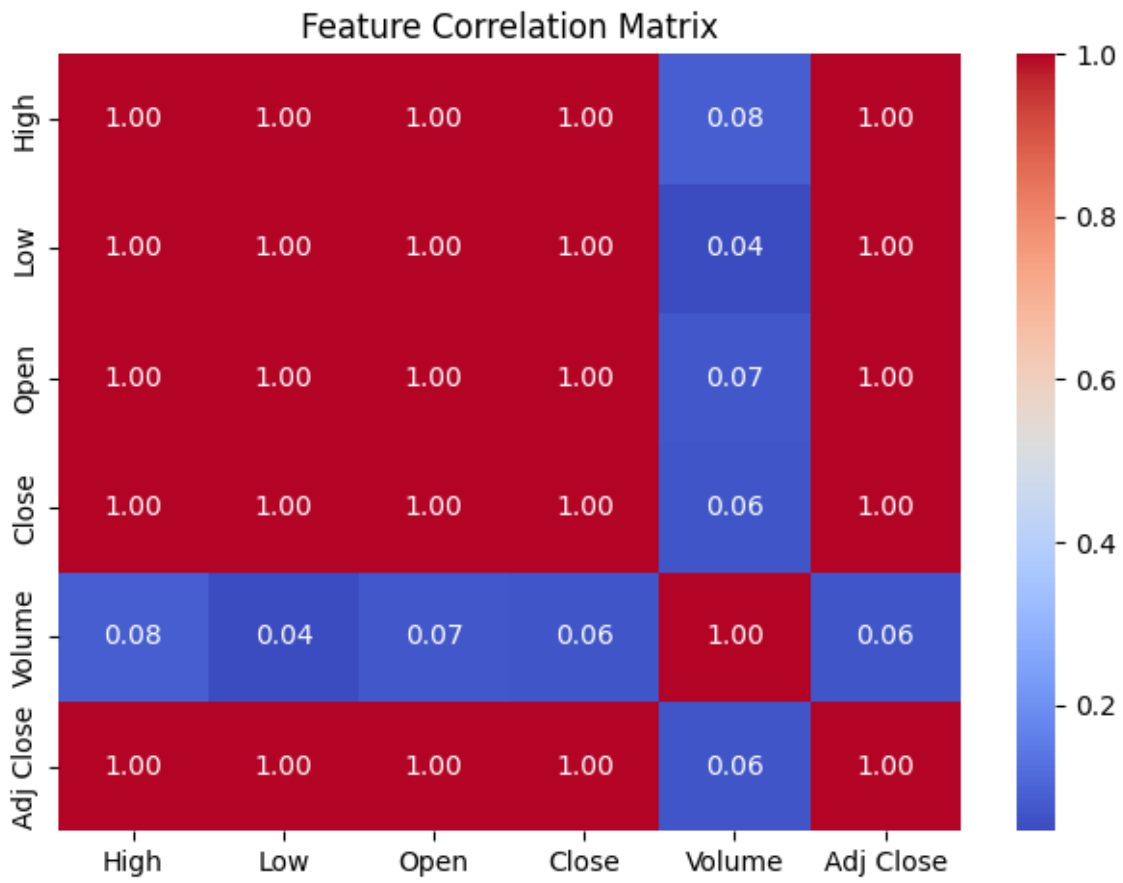
Tools and Technologies

Tool	Use
WSL2	To utilize Linux on windows for compatibility with python libraries and TensorFlow.
TensorFlow	To build the architecture of the LSTM neural network.
NumPy	For array manipulation and data processing.
Scikit-Learn	For correlation analysis and to output accuracy metrics for the model. Also used to train the model.
Pandas	For reading historical data for analysis and training.
Seaborn	For generating visualizations of the data and model metrics.
GitHub	For storing files and act as version control for the project.

Outputs and Visuals

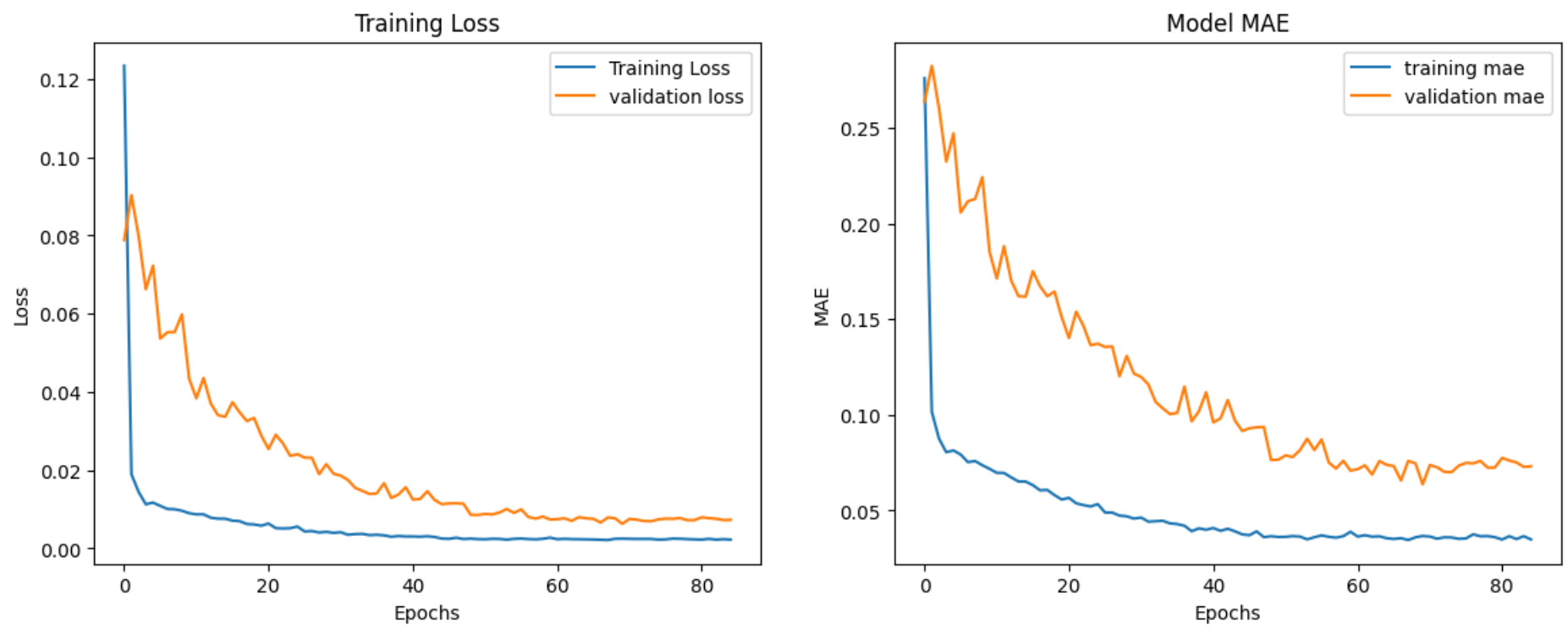


EDA of the historical stock price data before scaling, encoding and being sent to the model for training.

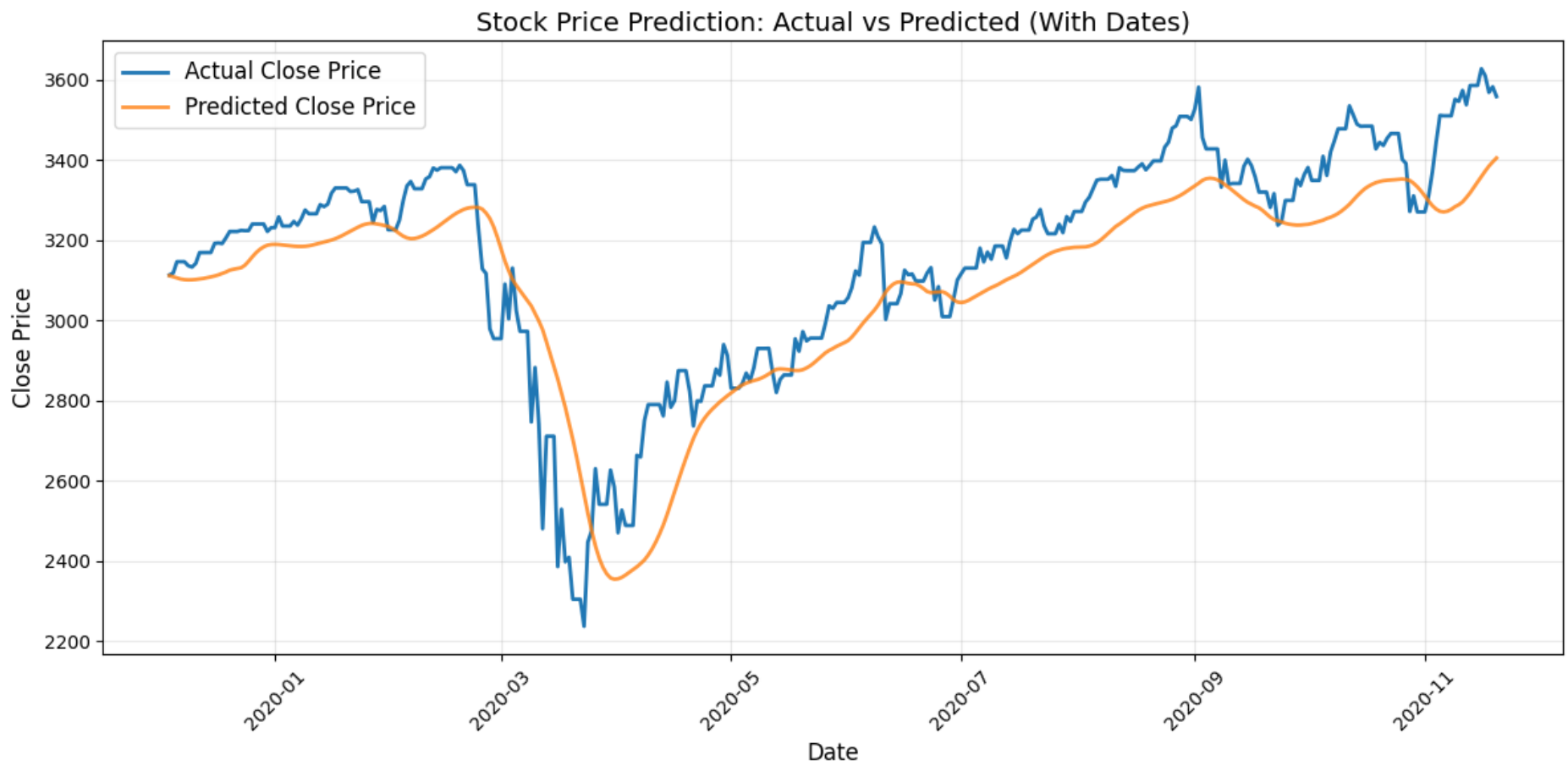


Correlation analysis of the dependent variables (high, low, open, close prices, volume) to the independent variable (Adjusted Close price).

Checking the historical data for seasonal trends.



Visual output of training and validation loss and MAE over the number of epochs.



Model predictions graphed against validation data to show the accuracy of the model.

Challenges and Solutions

Challenge	Solution
Determining the seasonality of the data and picking an appropriate look back value.	I used trail and error to pick a look back value of 60 days (two months). This seemed to provide a good increase to the accuracy of the model. For seasonality I used the Statsmodel library, which had many built-in tools to help decompose the data and provide the information required.
Choosing the number of layers and the underlying architecture of the neural network.	Again, the solution was a bit of trial and error. Building and testing the model quickly with different layers and outputting accuracy metrics to determine the best trade off between accuracy and training time.

Outcomes and Learnings

I was able to successfully build the model which ended up having an R2 score of .73. The model does lag behind the actual stock price by roughly 15-20 days so there is room for improvement. I think with some hyperparameter tuning this gap could be reduced, which should increase the R2 score. This project gave me hands-on experience with building an LTSM model in a real-world environment (stock price, finance). It also allowed me to continue to refine my programming skills, along with critical thinking and problem solving.

Creating a Real Time Machine Learning Model

Overview

The goal of this project was to create an end-to-end machine learning project using real time data. This data was streamed from a source using an API and Kafka. This data was then fed into a SGDRegressor model and the outputs from the model were displayed on a dashboard that was created using Streamlit.

Methodologies Used

Data Pipelines

The data from the API source is read and fed into the model the same way every time to ensure the model is trained the same way each time. There is a script to initialize the Kafka topics, to set up and start the producer, to set up and start the consumer, and for the model itself. The pipeline performs feature engineering and scaling of the dependent variables in the same way for model training.

Containerization

To properly set up a local Kafka instance, a docker container was utilized to ensure compatibility with my host system.

Logging

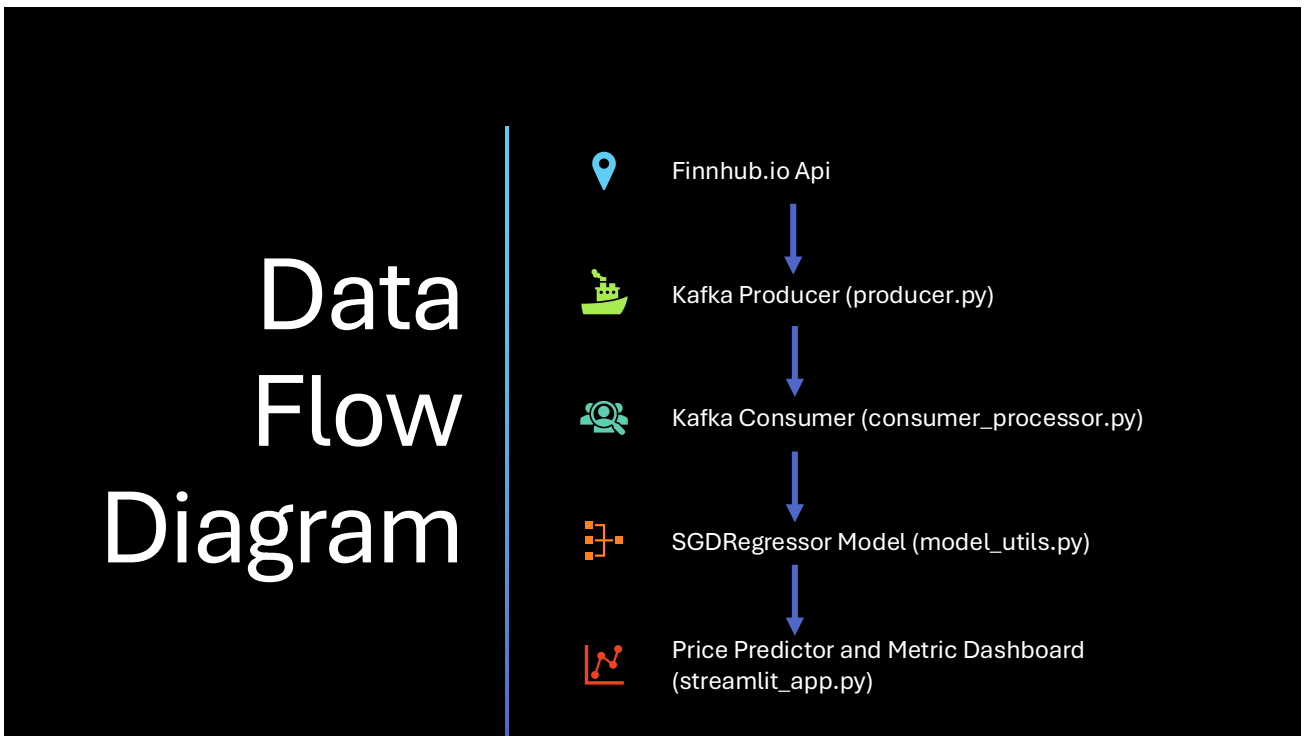
The application uses a logging function throughout to ensure the working condition of the application and for continuous monitoring.

Tools and Technologies

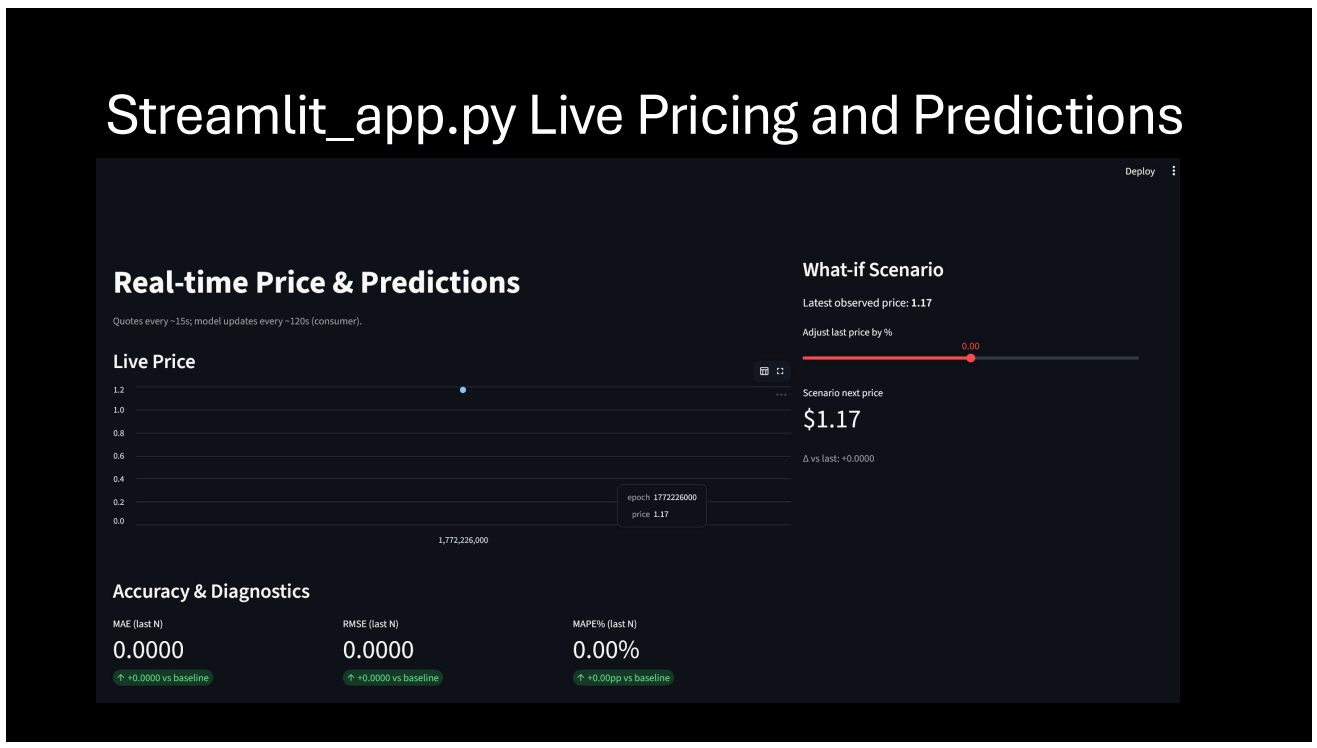
Tool	Use
WSL2	To utilize Linux on windows for compatibility with python libraries.
Kafka	To stream the data from the API source in real time.
NumPy	For array manipulation and data processing.

Scikit-Learn	For correlation analysis and to output accuracy metrics for the model. Also used to train and build the SGDRegressor model.
Pandas	For reading the data for analysis and training.
Seaborn	For generating visualizations of the data and model metrics.
Streamlit	For building the real time price prediction dashboard.
GitHub	For storing files and act as version control for the project.

Outputs and Visuals



The flow of the data through the various technologies that were used in this application.



The main page of the dashboard showing the actual stock price, the predicted price, and accuracy metrics of the ML model.

Screenshots from the terminal showing logging outputs from the Kafka producer and consumer.

Challenges and Solutions

Challenge	Solution
Displaying predictions and accuracy metrics from the predictor model in a logical concise way.	I created a mock dashboard to iteratively design the UI to ensure that everything was put in a good spot.
Ensuring the producer is writing data and the consumer is properly ingesting data.	To combat this logging was implemented at the consumer step to visually show that data was being ingested the way the application intended it.

Outcomes and Learnings

I was able to successfully ingest data from Finnhub.io using an API and Kafka, push it to a machine learning model for training, and then push price predictions to a dashboard using Streamlit. This project allowed me to get experience working with industry standard tools like Streamlit, Docker, Kafka, GitHub, and Scikit-Learn. It gave me the opportunity to think about a project from end to end and work on back and front end development. It also allowed me to continue to build on my programming and machine learning model building and scripting skills.